

In-Class Exercise 3

Client-Server Interaction with Node.js and Express

The goal of today's in-class activity is to help you understand the basics of client-server interaction using **Node.js** and **Express**. To complete this activity, you will:

- Set up a simple server using **Node.js** and **Express**
- Handle basic **HTTP requests** (GET and POST)
- Use **Postman** to send requests and view the server's responses

Setup and Installation

Step 1: Install Node.js and npm

- Download and install the latest LTS version from nodejs.org
- Verify installation:

```
node -v  
npm -v
```

Step 2: Initialize a New Node.js Project

```
mkdir <your-proj-name>  
cd <your-proj-name>  
npm init -y
```

Step 3: Install Express

```
npm install express
```

Step 4: Install Nodemon for Automatic Restart

During development, it is inconvenient to stop and restart the server manually every time you edit `server.js`. `nodemon` monitors your project files and automatically restarts the server when changes are saved

- Install `nodemon` as a development dependency:

```
npm install --save-dev nodemon
```

Step 5: Install Postman

Postman is a tool for sending HTTP requests to a server and inspecting the responses. It is useful for testing routes such as GET and POST without needing to build a frontend first.

- Preferred option: use the **Postman desktop app** for this exercise
- You may also use the **Postman web app**, but if you do, install and run the **Postman Desktop Agent** so requests to `http://localhost:3000` work correctly
- Download and install from postman.com

Configuring `package.json` and Creating the Server

When you ran:

```
npm init -y
```

Node.js created a `package.json` file in the root of your project folder.

Before creating `server.js`, update `package.json` for this exercise as instructed below.

1. Enable ES module syntax

This exercise uses modern JavaScript (**ES6+**) module syntax such as:

```
import express from "express";
```

By default, Node.js expects `.js` files to use **CommonJS** syntax, for example:

```
const express = require("express");
```

- To allow `import` syntax in this project, add the following property to `package.json`:

```
"type": "module"
```

2. Add a development script for nodemon

We will use nodemon to restart the server automatically whenever `server.js` is updated.

- Add the following script to `package.json`:

```
"scripts": {  
  "dev": "nodemon server.js"  
}
```

Example package.json:

```
{
  "name": "your-proj-name",
  "version": "1.0.0",
  "type": "module",
  "main": "server.js",
  "scripts": {
    "dev": "nodemon server.js"
  }
}
```

3. Implement server.js

Now create a file named **server.js** and write the given code to set up your Express server.

Note that the **POST /messages** route is intentionally left incomplete. You are expected to complete the **// TODO** section so the server reads the submitted JSON data, adds the current date, and returns the required JSON response.

```

// Import the Express module using modern JavaScript ES module syntax
import express from "express";

// Create an instance of an Express application
const app = express();

// Define the port number the server will listen on
const PORT = 3000;

/**
 * Express middleware for parsing JSON request bodies into req.body.
 * This allows the server to automatically read and parse JSON data
 * from the body of incoming HTTP requests (such as POST requests).
 * Without this, req.body would be undefined when handling JSON payloads.
 */
app.use(express.json());

/**
 * Route: GET /
 * Description: Handles requests to the root URL.
 * Response: Sends back a classic "Hello, World!" message.
 */
app.get("/", (req, res) => {
  res.send("Hello, World! Server is up and running.");
});

/**
 * Route: GET /greeting
 * Description: Handles requests to /greeting
 * Response: Sends back a JSON object with a greeting message.
 */
app.get("/greeting", (req, res) => {
  res.json({ message: "Hey there, this is your server responding!" });
});

/**
 * Route: POST /messages
 * Description: Handles POST requests to /messages
 * Request Body: JSON data sent by the client, available in req.body
 * Response: Send back a JSON object containing the submitted data and the
 *           current date. The response should have the form { received: ... }
 */
app.post("/messages", (req, res) => {
  // TODO: create a new object that copies req.body, adds the current date
  // using the JS Date object, and send it back inside a "received" field
});

/**
 * Start the server. The server listens on the defined PORT and logs a
 * message to the console when running.
 */
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});

```

Running the Server

Make sure your `package.json` includes the required updates described earlier.

Then start the server:

```
npm run dev

# If you are not using nodemon:
node server.js
```

Once running, your server will be available at:

<http://localhost:3000>

Sending Requests with Postman

Before Testing:

Make sure your server is **running** in your terminal with:

```
npm run dev
```

Confirm the server output says:

```
Server is running on http://localhost:3000
```

Open Postman and create a new request for each route.

For **POST** requests:

- Select the **Body** tab
- Choose **raw**
- Change the format from **Text** to **JSON** using the dropdown
- Enter your JSON data properly formatted using `{ }` and standard double quotes

When you choose **raw** → **JSON**, Postman will usually set the correct header automatically. Verify that the following header is present:

```
Content-Type: application/json
```

Testing the Server with Postman

GET Request to /

Method: **GET**
URL: http://localhost:3000/
Response: "Hello, World! Server is up and running."

GET Request to **/greeting**

Method: **GET**
URL: http://localhost:3000/greeting
Response:
{
 "message": "Hey there, this is your server responding!"
}

POST Request to **/messages**

Method: **POST**
URL: http://localhost:3000/messages
Body (raw JSON): {
 "title": "My First Message",
 "content": "Today I started learning about Node and Express!"
}

Response:
{
 "received": {
 "title": "My First Message",
 "content": "Today I started learning about Node and Express!",
 "date": "2026-03-23"
 }
}

Note: The date value shown above is only an example. In your program, it should be set automatically to the current date when the request is processed.

Submitting your Solution

Submit a **zipped project folder** (<*first name*>_<*last name*>_inclass3.zip) containing only the files needed to run and review your solution.

Include:

- `server.js`
- `package.json`
- `package-lock.json` (if generated)

Do **not** include:

- `node_modules/`
- `.DS_Store`
- other automatically generated system files

Your project should run successfully after the grader executes:

```
npm install
node server.js
```